



StoreRate

ROXILER SYSTEMS · FULLSTACK INTERN CODING CHALLENGE

StoreRate

A store rating platform with role-based access.
Project overview, architecture and deployment
reference.

LIVE APP

roxiler-storerate-rho.vercel.app

SOURCE CODE

github.com/Georgian-86/Roxiler

AUTHOR

Georgian-86

VERSION

1.0

DATE

5 October 2026

STACK

React · Express ·
PostgreSQL

Executive summary

StoreRate lets users rate the stores registered on the platform on a scale of 1 to 5. Everyone signs in through the same login page. What each person can see and do depends on their role: **System Administrator**, **Normal User** or **Store Owner**.

The app has three parts: a React single-page frontend, an Express REST API and a PostgreSQL database. In production the frontend and API are hosted together on Vercel and the database runs on Supabase, both in Mumbai. The same form rules are checked in the browser, by the API and by database constraints. Automated tests check the deployed app before release.

3

user roles, one login

58

automated unit & integration tests

49

end-to-end checks run against production

Contents

1	Quick access	Links and demo credentials
2	Features	Capabilities per role, validation rules
3	Architecture	System design, request lifecycle, code structure
4	Data model	Schema, relationships, integrity rules
5	Security	Authentication, authorisation, data protection
6	Infrastructure & identifiers	Vercel, Supabase, GitHub, environment
7	Quality assurance	Test suites and results
8	Operations	Running locally, deploying, rotating secrets
A	API reference	REST endpoints

Quick access

Links

RESOURCE	URL
Live application	https://roxiler-storerate-rho.vercel.app
Health check (API + database)	https://roxiler-storerate-rho.vercel.app/api/health
GitHub repository	https://github.com/Georgian-86/Roxiler
Continuous integration	https://github.com/Georgian-86/Roxiler/actions
Vercel project dashboard	https://vercel.com/golus-projects/roxiler-storerate
Supabase project dashboard	https://supabase.com/dashboard/project/rkjahfmbmsylelbmhlru

Demo credentials

All demo accounts use the password `Password@123`. You can also create a new Normal User from the Sign up page.

ROLE	EMAIL	OPENS ON
Administrator	admin@roxiler.com	Dashboard, Users, Stores
Normal User	user@roxiler.com	Browse & rate stores
Normal User	rahul@roxiler.com	Browse & rate stores
Store Owner	owner@roxiler.com	Sunrise Organic Grocery Market
Store Owner	owner2@roxiler.com	Metro Electronics Superstore

Suggested walkthrough. Sign in as `user@roxiler.com`, then search, sort and rate a store. Next, sign in as `admin@roxiler.com` to see the updated totals, filter the user list and add a store. Finally, sign in as `owner@roxiler.com` to see the new rating on the owner dashboard.

Features

System Administrator

- Dashboard showing total users, stores and ratings
- Add Normal, Admin and Store Owner users
- Add stores; assign or change a store's owner
- Store list: name, email, address, rating, owner
- User list: name, email, address, role, rating
- Filter by name, email, address and role
- User details, including the store's rating for owners
- Change password · log out

Normal User

- Sign up and log in
- Browse every registered store
- Search by store name and address
- See each store's overall rating and your own rating
- Submit a 1–5 star rating
- Change a rating you already submitted
- Change password · log out

Store Owner

- Log in to a dashboard for their own store
- Average rating and total number of ratings
- 1–5 star breakdown chart
- List of customers who rated the store, with their rating and date
- Change password · log out

Shared behaviour

Tables

Every table can be sorted ascending or descending on its main columns. Sorting, filtering and paging are done by the server, and text filters wait for the user to pause typing before searching.

Experience

Responsive layout (tables become cards on phones), automatic dark mode, keyboard-accessible star ratings and dialogs, loading placeholders, and confirmation messages.

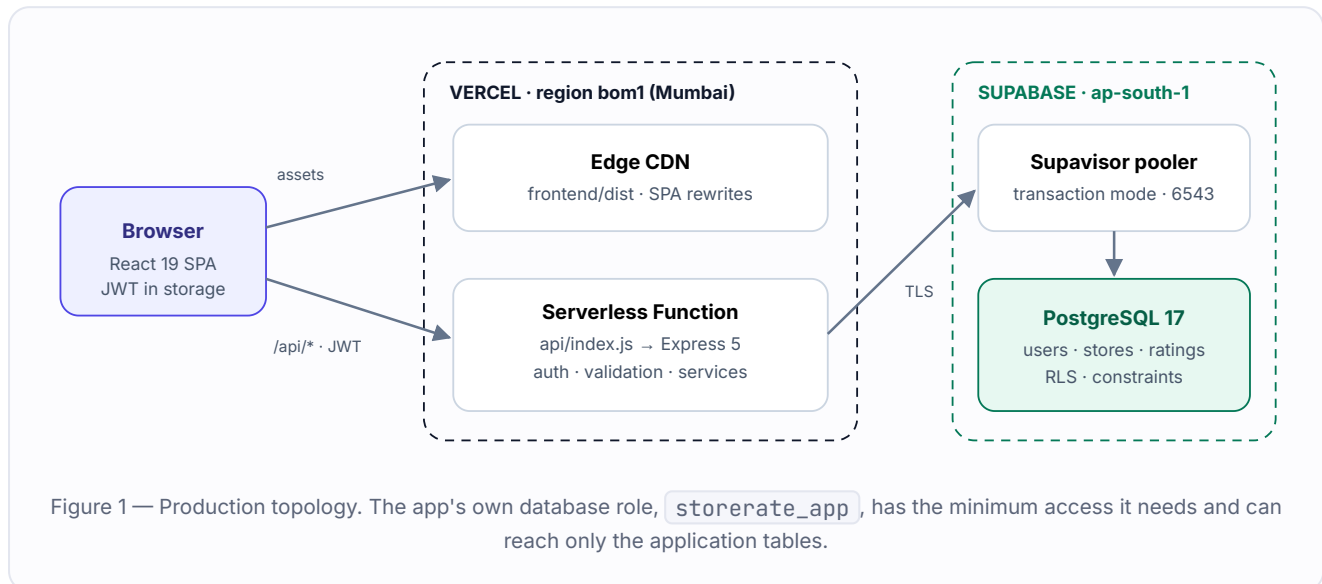
Validation rules

These rules are checked in three places: in the browser for instant feedback, by the API as the authoritative check, and by database constraints as a final safeguard.

FIELD	RULE
Name	20 to 60 characters (user and store names)
Address	Required, at most 400 characters
Password	8 to 16 characters, including at least one uppercase letter and one special character
Email	Standard email format; unique regardless of letter case
Rating	Whole number from 1 to 5; one rating per user per store

Architecture

A single Vercel project hosts the whole app. The React build is served as static files from Vercel's CDN. Every `/api/*` request goes to one serverless function that runs the Express app, and that function connects to Supabase Postgres through Supabase's connection pooler.



Frontend

React 19, Vite and React Router 7. Axios adds the login token to every request and logs the user out if the token is rejected. Routes are guarded by role.

Backend

Express 5 with `express-validator`, JWT and `bcrypt`, organised as routes → middleware → services. Database access uses parameterised SQL through `pg`.

Database

PostgreSQL with plain SQL migrations, case-insensitive email columns (`citext`), CHECK and UNIQUE constraints, and timestamps kept up to date by triggers.

Request lifecycle — a user rates a store

- 1 The browser sends `PUT /api/stores/12/rating` with body `{"rating": 4}` and the user's login token.
- 2 The `authenticate` middleware verifies the token, reloads the user from the database, and rejects tokens issued before the user's last password change.
- 3 The `authorize('USER')` check lets only Normal Users continue.
- 4 The `validate` step checks that the rating is a whole number from 1 to 5. If not, the API returns 400 with an error for that field.
- 5 `storeService.upsertRating` creates the rating, or replaces the user's earlier one, in a single SQL statement. It responds 201 for a new rating or 200 for an update, along with the store's new average.
- 6 The React table updates the row in place. If the list is sorted by rating, it reloads so the order stays correct.

Code organisation

backend/src

routes/	auth, admin, stores, owner endpoints
middleware/	login check and role guard, validation, error handling
validators/	form rules shared by all endpoints
services/	SQL queries and business rules
utils/	error type, safe sort, filters, pagination
db/	connection pool, migrations, seed script

frontend/src

api/	HTTP client that adds the login token
context/	current user and notifications
hooks/	useForm (validation), useList (filter, sort, pages)
components/	DataTable, FilterBar, Modal, StarRating, Layout
pages/	auth, admin/, user/, owner/
utils/	validation rules (same as the backend), constants

Data model

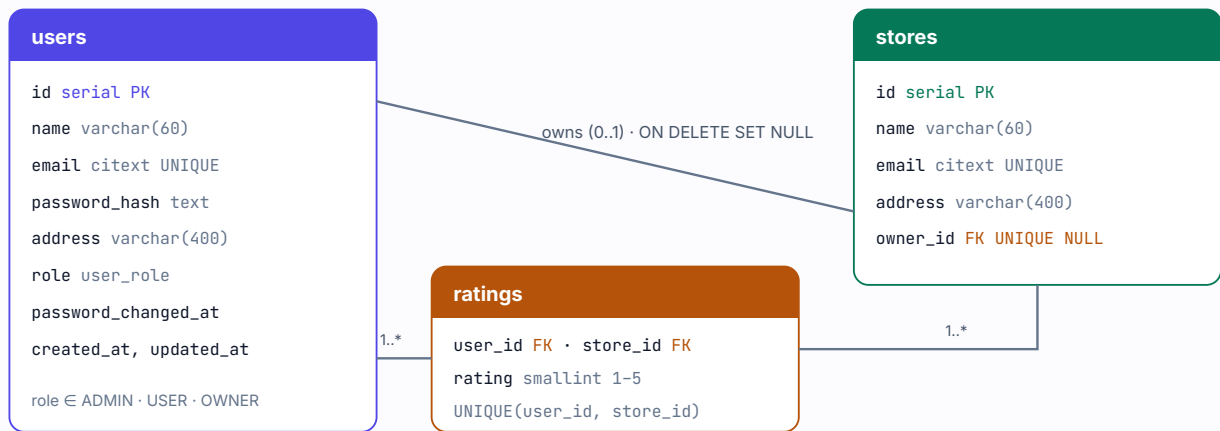


Figure 2 — Entity relationships. Deleting a user or store also deletes their ratings.

Integrity rules

- A unique pair of user and store allows exactly one rating each; a second rating replaces the first
- CHECK constraints repeat the name, address and rating rules, so bad data is rejected even if it bypasses the API
- A unique `owner_id` gives each owner at most one store
- Emails are compared without regard to letter case, so duplicates such as `A@x.com` and `a@x.com` are rejected

Design choices

- Averages are calculated with `AVG()` each time they're read, so they can never fall out of step with the ratings
- Index on `ratings(store_id)` to speed up averages and the owner dashboard
- `updated_at` timestamps maintained by triggers
- Versioned SQL migrations, each applied in a transaction and recorded in `schema_migrations`

Security

Authentication

- Passwords hashed with bcrypt
- Login takes the same time whether or not the email exists, and always gives the same error message
- Login tokens (JWT) expire after one day
- Changing a password signs out older sessions; the current session gets a fresh token
- Login and sign-up are rate-limited to slow down password guessing

Authorisation

- The user's role is reloaded from the database on every request, so a role change takes effect immediately
- The server enforces role checks on every endpoint; the frontend checks only decide what is shown
- Public sign-up always creates a Normal User; a role sent with the request is ignored

Input handling

- All SQL uses parameters, and sort columns come from a fixed allowed list
- The `%` and `_` wildcards are treated as plain text in search filters
- Request bodies are limited to 10 KB; database errors are turned into clear 400 or 409 responses

Database (Supabase)

- The app connects with its own `storeRate_app` role, which can only read and write the app tables
- Row-level security is enabled, and the anonymous and signed-in roles of Supabase's public Data API have no access
- Connections are encrypted with TLS; Supabase's security advisor reports no issues

Secrets. The database password and token signing secret are stored only as sensitive environment variables in Vercel and in a private local file (`SECRETS.local.md` , not committed to git). They are intentionally not included in this document.

Infrastructure & identifiers

Vercel

Team	Golus's projects
Team slug	golus-projects
Team ID	team_0m9MRcEqM9bCQuQ3NoHwhurG
Project	roxiler-storerate
Project ID	prj_1lCRsHEw74t0bGiAsjU1uwQLB0sk
Region	bom1 (Mumbai)
Deploys from	GitHub <code>main</code> (each push deploys automatically)

Supabase

Organisation ID	flkoimkmgkbizjubynca
Project	roxiler-storerate
Project ref	rkjahfmbsmylelbmhlru
Region	ap-south-1 (Mumbai)
Engine	PostgreSQL 17
Pooler	aws-0-ap-south-1.pooler.supabase.com:6543
App role	storerate_app

GitHub

Repository	Georgian-86/Roxiler
Production branch	<code>main</code>
Development branch	<code>claudes/epic-cori-o1qa2j</code>
CI workflow	<code>.github/workflows/ci.yml</code> : backend lint and tests against PostgreSQL 16; frontend lint, tests and production build

Environment variables (Vercel)

KEY	TYPE	PURPOSE
DATABASE_URL	Sensitive	Supabase pooler connection string for <code>storerate_app</code>
JWT_SECRET	Sensitive	Signing key for login tokens
DATABASE_SSL	Plain · <code>true</code>	Encrypt the connection to Supabase
DB_POOL_MAX	Plain · <code>3</code>	Small connection pool per serverless instance
TRUST_PROXY	Plain · <code>1</code>	Use the visitor's real IP address for rate limiting

Quality assurance

Each change is checked at four levels: integration tests against a real database, frontend unit tests, CI on every push, and end-to-end smoke tests against the live deployment, both through the API and in a real browser.

SUITE	SCOPE	RESULT
Backend integration Jest + Supertest	Authentication, validation, role access, filters, sorting and paging, rating create and update, owner dashboard, revoking old sessions	53 / 53 passed
Frontend unit Vitest	Validation rules, matching the backend	5 / 5 passed
Production API smoke test scripts/smoke-test.mjs	Infrastructure, authentication, each role's actions and the password-change flow, run against the live URL	34 / 34 passed
Production browser test scripts/ui-smoke-test.mjs	Playwright, Chromium: sign up, rate, search, sort, change password, admin and owner flows, phone navigation	15 / 15 passed
Static analysis	ESLint (backend), oxlint (frontend), Supabase security advisor	Clean

An independent code review was carried out against the challenge specification before release. Its findings were fixed and covered by new tests, including email handling, timing-safe login, revoking old sessions, accurate conflict errors, pagination edge cases and reassigning store owners.

Operations

Run locally

```
# 1. Database (Postgres 16 on :5432, creates roxiler and roxiler_test)
docker compose up -d db

# 2. API on http://localhost:4000
cd backend && cp .env.example .env && npm install
npm run seed && npm run dev

# 3. Web app on http://localhost:5173 (sends /api requests to the local API)
cd ../frontend && npm install && npm run dev
```

Deploy

- 1 Push to `main`. Vercel installs, builds the frontend, packages the API function and puts the new version live.
- 2 Schema changes go in a new file under `backend/src/db/migrations/`. Apply it to Supabase *before* deploying code that depends on it.
- 3 Check the release:
`BASE_URL=https://roxiler-storerate-rho.vercel.app node scripts/smoke-test.mjs`

Rotate secrets

Database password

Run `ALTER ROLE storerate_app PASSWORD '...'` in the Supabase SQL editor, update `DATABASE_URL` in Vercel, then redeploy.

JWT secret

Set a new `JWT_SECRET` in Vercel and redeploy. Everyone will need to log in again.

API reference

All endpoints are under `/api`. Authenticated endpoints need the header `Authorization: Bearer <token>`. List endpoints accept `sortBy`, `order` (asc or desc), `page` and `limit`, and return `{ data, meta }`.

METHOD	PATH	ACCESS	DESCRIPTION
GET	/health	Public	API and database status
POST	/auth/register	Public	Sign up (always creates a Normal User)
POST	/auth/login	Public	Returns a login token and the user's profile
GET	/auth/me	Any role	The signed-in user
PATCH	/auth/password	Any role	Change password; returns a new token
GET	/admin/dashboard	Admin	Total users, stores and ratings
GET	/admin/users	Admin	Filters: name, email, address, role
POST	/admin/users	Admin	Create a user of any role
GET	/admin/users/:id	Admin	User details, plus store and rating for owners
GET	/admin/owners/available	Admin	Store owners who don't have a store yet
GET	/admin/stores	Admin	Filters: name, email, address
POST	/admin/stores	Admin	Create a store, optionally with an owner
PATCH	/admin/stores/:id/owner	Admin	Assign, change or clear a store's owner
GET	/stores	User	Store list with overall and own rating; search by name or address
PUT	/stores/:id/rating	User	Submit (201) or change (200) a rating
GET	/owner/dashboard	Owner	Store, average, rating breakdown, customers who rated